

Validación cruzada

Ciencia de datos para la toma de decisiones I

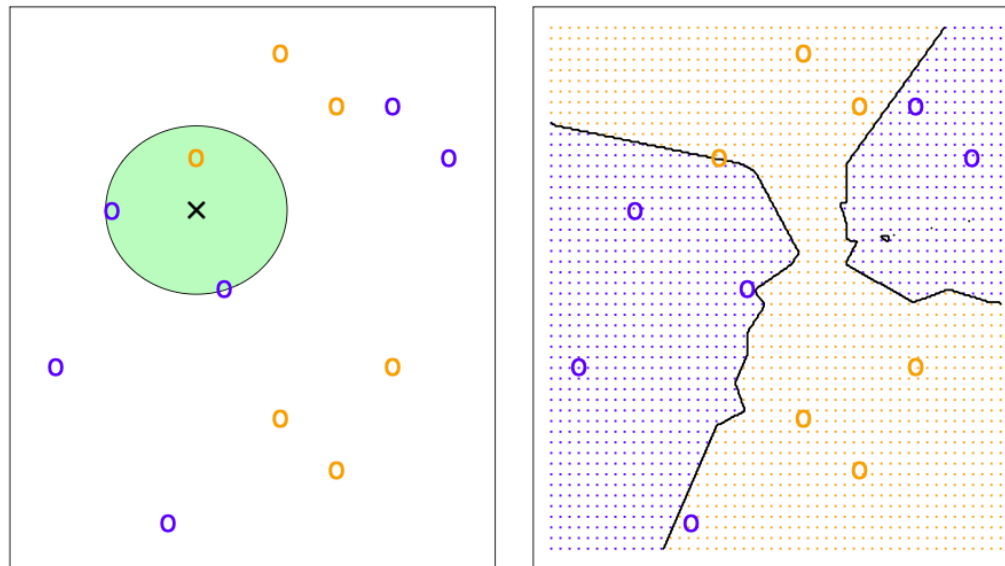
Jorge Juvenal
Campos
Ferreira

✉ juvenal.campos@tec.mx

K-vecinos cercanos

K-vecinos cercanos

El método de k-vecinos cercanos es un modelo de **clasificación supervisada**, basado en **distancia**, que permite clasificar observaciones nuevas basado en la proximidad que tenga un punto nuevo a clasificar con sus k vecinos más cercanos.



¿Que es K-NN?

Algoritmo de aprendizaje supervisado propuesto formalmente por Fix y Hodges (1951) y extendido por Cover y Hart (1967).

Doble proposito

- * **Clasificación:** asigna una categoria (p.ej. fraude / no fraude).
- * **Regresión:** predice un valor continuo (p.ej. precio de casa).

Lazy learner

No construye un modelo explicito durante el entrenamiento. Memoriza los datos y solo calcula al momento de predecir.

No parametrico

No asume ninguna forma de distribución de los datos. Esto le permite capturar fronteras de decisión muy flexibles, sin imponer supuestos como linealidad o normalidad.

Idea clave: K-NN predice mirando a los vecinos mas parecidos del nuevo punto en el conjunto de entrenamiento.

La intuición

"Dime con quien andas y te diré de qué clase eres."

Premisa

Si un nuevo punto **se parece** a sus vecinos en el espacio de características, probablemente pertenezca a su misma clase.

Idea central

Proximidad implica similitud.

Toda la mecánica del algoritmo se reduce a medir distancias y consultar a los vecinos.

Otro ejemplo

Piénsalo así

Si quieres adivinar si a alguien le gustará una película, preguntas a las personas más PARECIDAS a esa persona (misma edad, mismos gustos) y ves qué opina la mayoría.

KNN hace exactamente eso: para clasificar un caso nuevo, busca los casos más parecidos que ya conoce y copia la respuesta de la mayoría.

El espacio de características

$$\mathbf{x} = (x_1, x_2, \dots, x_n)$$

Cada observación es un vector con n atributos

Geometria

Cada observación es un punto en un espacio de n dimensiones. Con 2 variables: plano. Con 3: cubo. Con n : hipercubo.

Cercanía = metrica

"Cercanía" no es algo abstracto: **se mide con una formula de distancia** que asigna un numero a cada par de puntos.

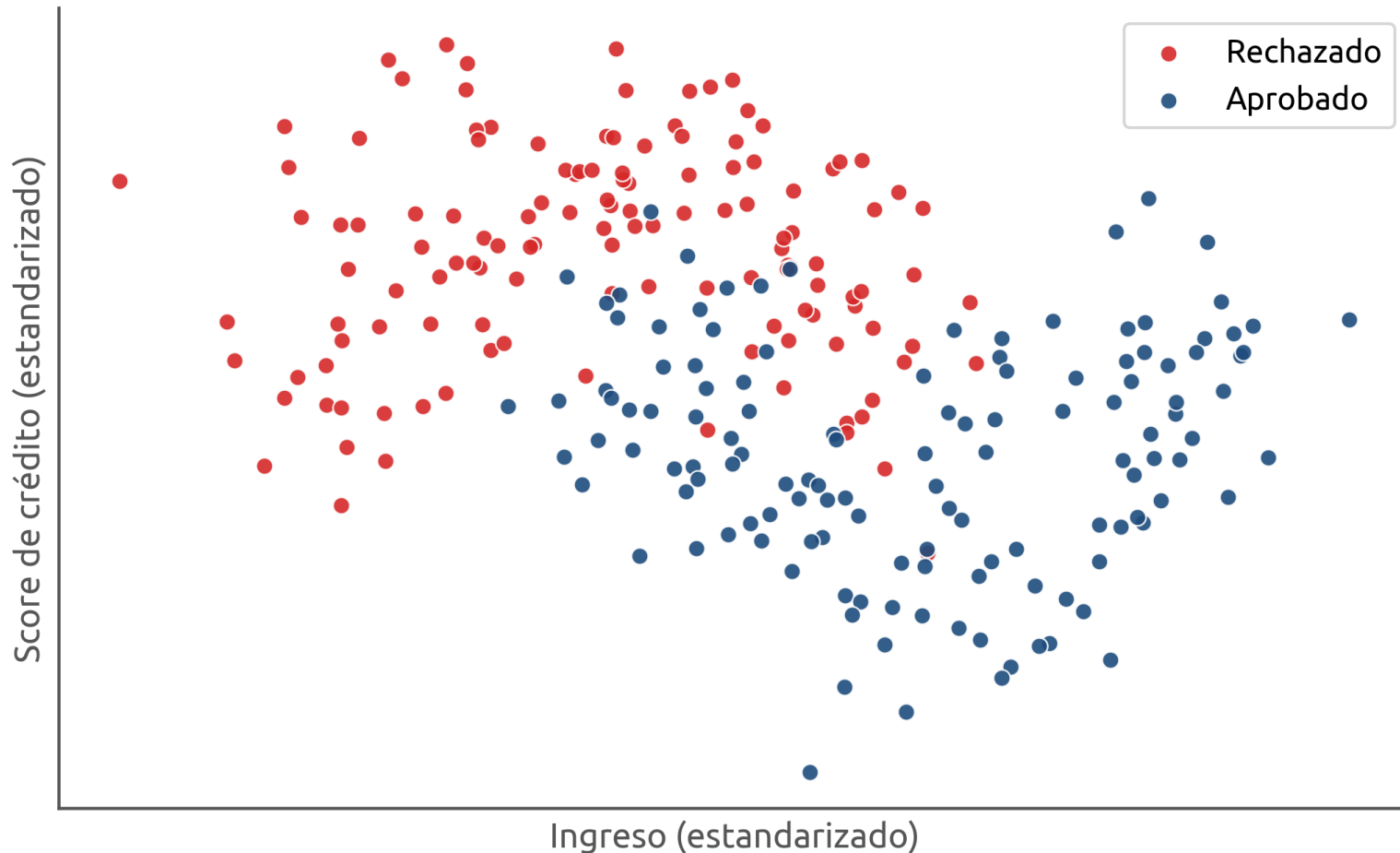
Ejemplo (3 variables)

Un municipio podría describirse con un vector (PIB per cápita, índice de pobreza, escolaridad promedio). Cada uno es un punto en $\mathbb{R}^3$ y municipios parecidos quedarían cerca entre si, aunque no estén cerca físicamente.

El espacio de características

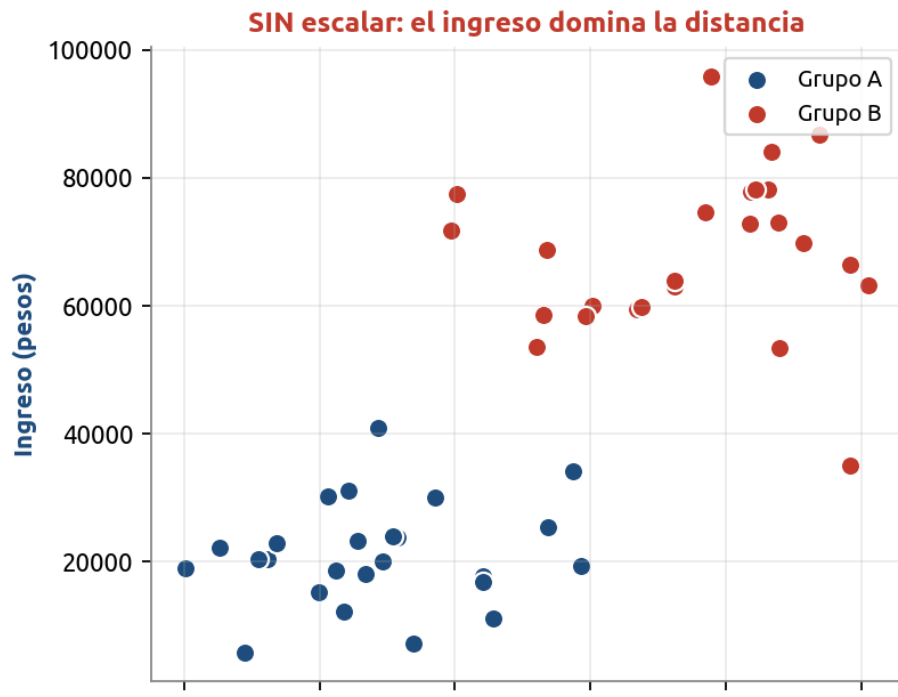
A continuación se muestra un **espacio de características** en R2 conformado por el eje X = ingreso y el eje Y = score de crédito.

Cada solicitud es un punto en el espacio de características



Escalar las variables es OBLIGATORIO

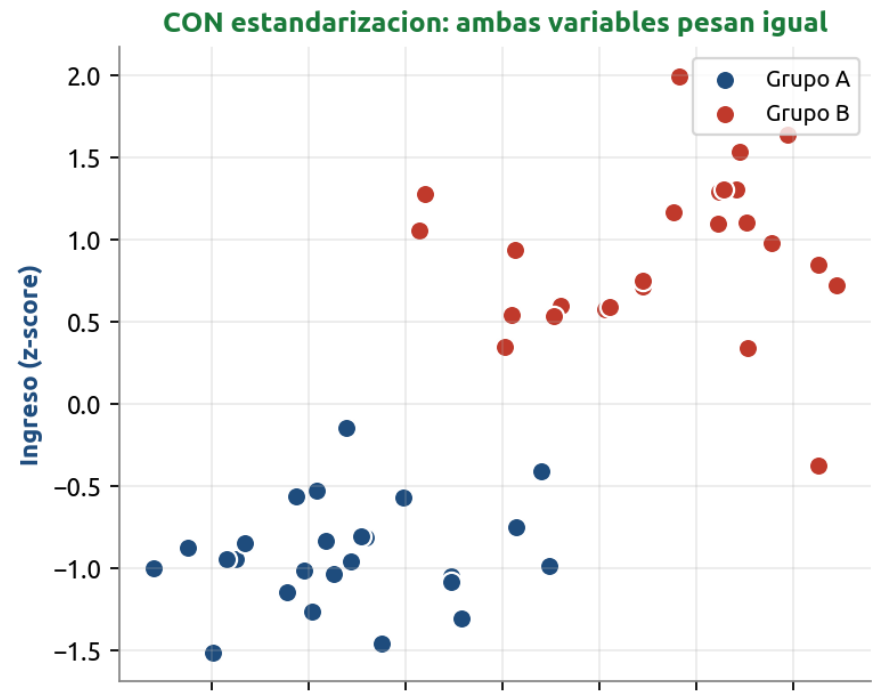
Las variables con escalas grandes dominan el calculo de distancia.



Estandarizacion (z-score)

$$x' = (x - \mu) / \sigma$$

Media 0 y desviacion 1.



Normalizacion min-max

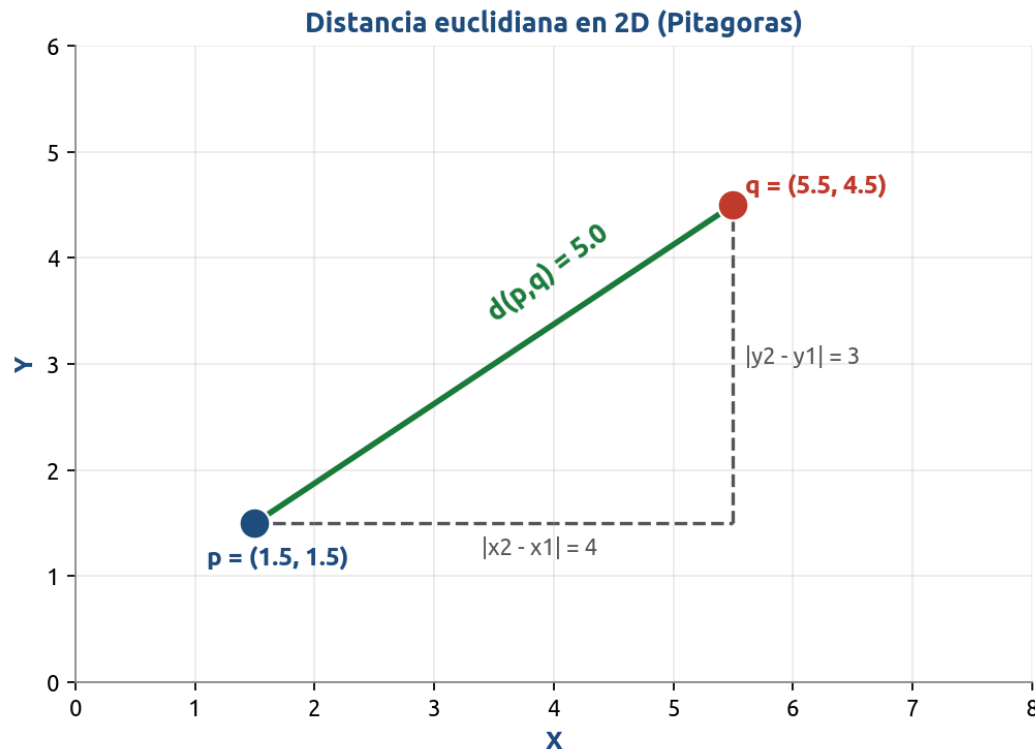
$$x' = (x - x_{\min}) / (x_{\max} - x_{\min})$$

Todo queda en [0, 1].

Distancia euclidiana (la mas usada)

$$d(p,q) = \sqrt{\sum_{i=1..n} (p_i - q_i)^2}$$

Generalizacion de Pitagoras a n dimensiones



Notacion

- * p y q : dos puntos en R^n
- * p_i, q_i : i -esima coordenada
- * n : numero de variables

Caso 2D (Pitagoras)

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Otras métricas de distancia

Metrica	Formula	Cuando usarla
Manhattan (L1)	$\sum p_i - q_i $	Variables tipo grilla, robusta a outliers
Minkowski	$(\sum p_i - q_i ^p)^{1/p}$	Generaliza L1 y L2 (parametro p)
Coseno	$1 - (p \cdot q) / (\ p\ \ q\)$	Texto, datos de alta dimensionalidad

Como elegir la metrica?

No hay una respuesta unica: depende del tipo de variables y del dominio. La euclidiana es el punto de partida por defecto; Manhattan ayuda con outliers; coseno funciona bien con texto.

Voto mayoritario (formalización)

Para clasificación, la predicción para un punto x es:

$$y(x) = \arg \max_{c \in \mathcal{C}} \sum_{i \in N_k(x)} 1(y_i = c)$$

Términos:

- x : el punto que queremos clasificar.
- $y(x)$: la clase predicha para x .
- $\mathcal{C}$: el conjunto de clases posibles.
- $N_k(x)$: los k vecinos más cercanos a x según una métrica de distancia.
- y_i : la clase verdadera del vecino i .
- $1(y_i = c)$: función indicadora — vale 1 si el vecino i es de clase c , 0 si no.
- $\sum 1(y_i = c)$: cuenta cuántos vecinos son de clase c .
- $\arg \max_c$: devuelve la clase con la cuenta más alta.

KNN paso a paso

Proceso: medir, ordenar y votar.

1

Llega un caso nuevo

Una solicitud sin clasificar todavía.

2

Medir distancias

Calculamos qué tan lejos está de cada caso conocido.

3

Elegir los k más cercanos

Nos quedamos con los k vecinos más parecidos (por ejemplo, $k = 5$).

4

Votar

La clase que más se repite entre esos k vecinos gana.

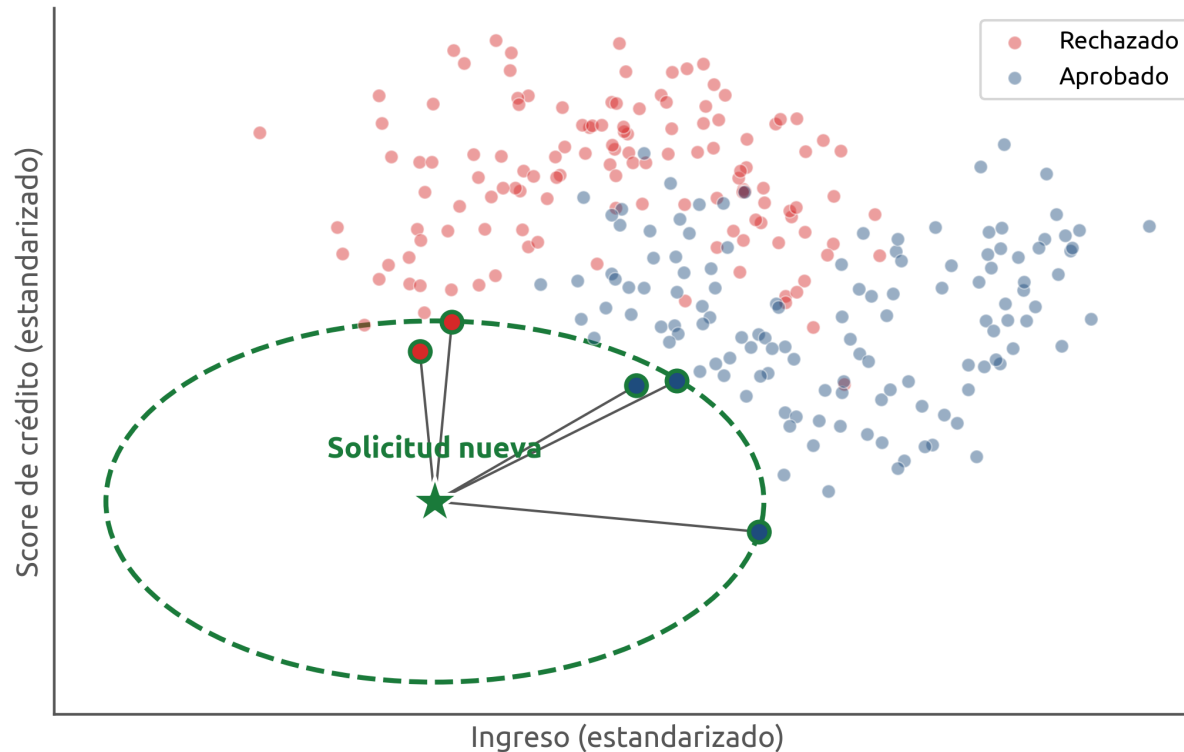
5

Asignar la clase

El caso nuevo recibe la clase ganadora.

La votación en acción

$k = 5$ vecinos más cercanos votan: 3 aprobado vs 2 rechazado



Con $k = 5$, ganan 3 votos de 'aprobado' sobre 2 de 'rechazado'.

El caso nuevo (estrella) se clasifica como '**aprobado**' porque la mayoría de sus 5 vecinos más cercanos están **aprobados**.

KNN es un modelo 'perezoso'

Definición

A diferencia de otros modelos, KNN no calcula fórmulas ni 'estudia' por adelantado: simplemente GUARDA todos los ejemplos. El trabajo lo hace hasta que llega un caso nuevo y toca medir distancias y votar.

En palabras simples

Es como un estudiante que no estudia antes, pero tiene todos sus apuntes a la mano y los consulta justo en el examen.

Ventajas y desventajas de K-NN

Conceptualmente simple

Fáciles de entender e interpretar: 'mira a tus vecinos y decide'. Útil para comunicar a audiencias no técnicas.

Sin fase de entrenamiento

No hay función de costo que optimizar ni parámetros que ajustar; basta con almacenar los datos.

Multi-clase natural

Maneja sin esfuerzo problemas con tres o más categorías: el voto mayoritario se generaliza directamente.

No paramétrico

No asume ninguna distribución subyacente; se adapta a la estructura real de los datos.

Fronteras no lineales

Captura fronteras de decisión arbitrariamente complejas sin tener que especificarlas a priori.

Versátil

Sirve tanto para clasificación como para regresión, con la misma lógica subyacente.

Costoso en predicción

Complejidad $O(n \times d)$ por punto a predecir: se vuelve lento con muchos datos o muchas variables.

Maldición de la dimensionalidad

En alta dimensión todas las distancias se vuelven similares y el concepto de 'vecino cercano' pierde sentido.

Requiere mucha memoria

Hay que guardar todo el conjunto de entrenamiento; no hay un modelo compacto que reemplace los datos.

Sensible a escalas

Una variable mal escalada puede dominar el cálculo y arruinar las predicciones.

Variables irrelevantes danan

K-NN no distingue por sí solo variables informativas de ruido: todas suman a la distancia.

Sensible al desbalance

Clases con muchos ejemplos tienden a 'ganar' los votos, sesgando la predicción contra las minoritarias.

Variantes y mejoras

Weighted K-NN

Cada voto se pondera por el inverso de la distancia ($w_i = 1/d_i$). Los vecinos mas cercanos influyen mas que los lejanos, reduciendo el impacto del ruido.

KD-Trees / Ball-Trees

Estructuras de datos que permiten buscar a los k vecinos en tiempo sub-lineal en lugar de comparar contra todos los puntos. Muy efectivas en dimensionalidad moderada.

Approximate Nearest Neighbors (ANN)

Algoritmos aproximados que sacrifican exactitud por velocidad: FAISS, Annoy, HNSW. Permiten K-NN sobre millones o miles de millones de puntos.

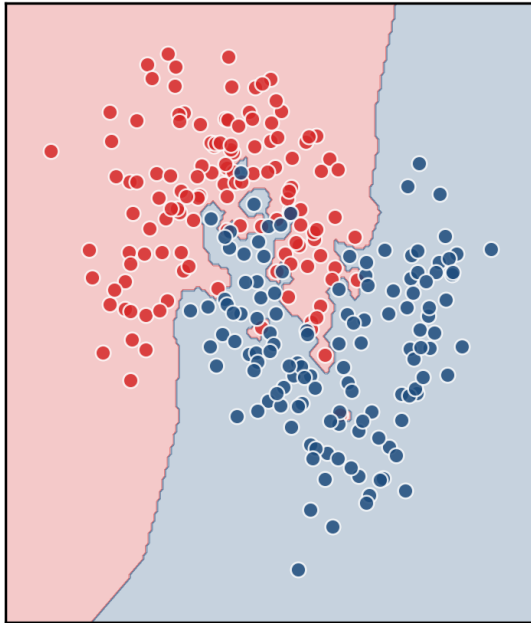
Reducción de variables previa

Aplicar PCA, selección por feature importance u otras técnicas para reducir la dimensionalidad ANTES de K-NN, mitigando la maldición de la dimensionalidad.

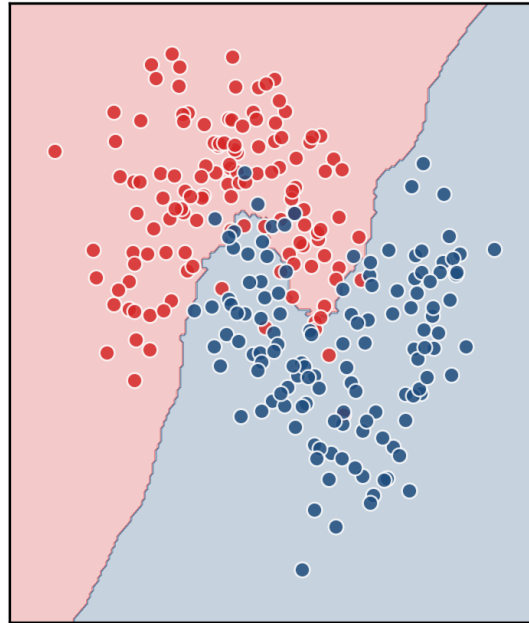
Casi todas las mejoras atacan dos problemas: velocidad de búsqueda o calidad de la métrica en alta dimensión.

La frontera según k

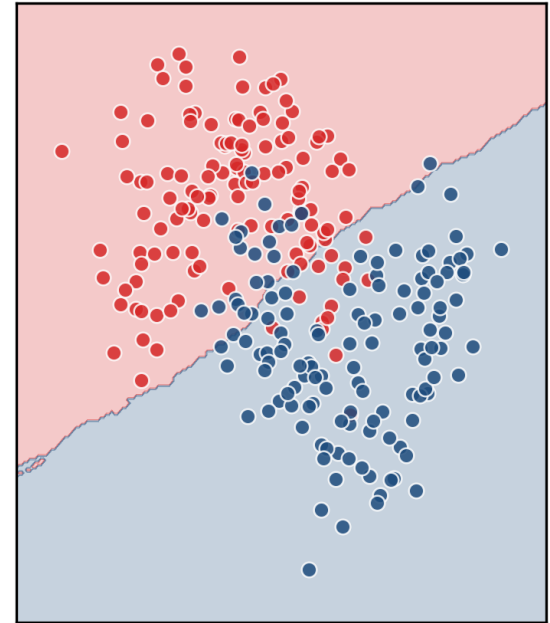
k = 1 (sobreajuste)



k = 15 (equilibrado)



k = 120 (subajuste)



Misma información, tres valores de k: la frontera cambia mucho.

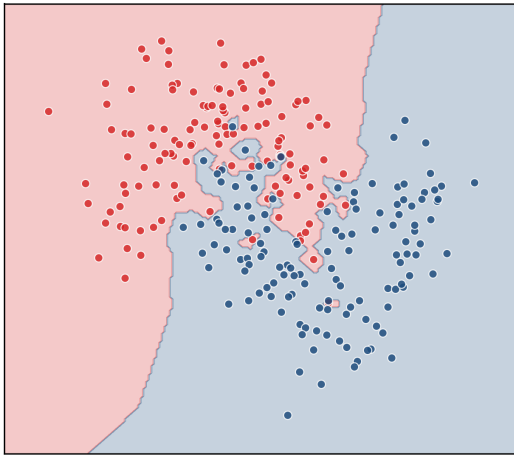
Con $k = 1$ la frontera es muy retorcida; con $k = 120$ es casi una línea recta. En medio está el equilibrio.

k pequeño vs k grande

Los dos extremos son malos, por razones opuestas.

k = 1: sobreajuste

k = 1: frontera ruidosa

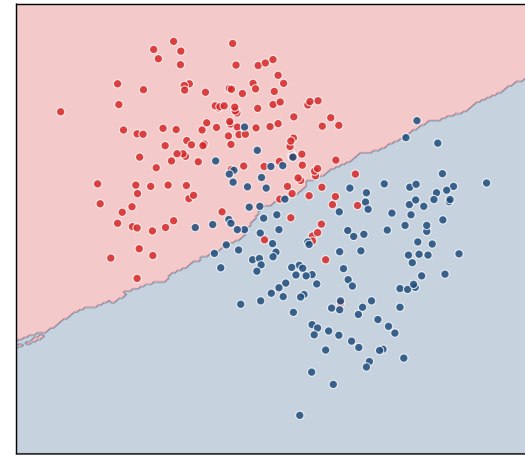


Le hace demasiado caso a CADA punto, hasta al ruido.

- Frontera muy retorcida
- Memoriza en vez de generalizar
- Falla con datos nuevos

k = 120: subajuste

k = 120: frontera demasiado plana



Promedia tantos vecinos que borra el detalle real.

- Frontera demasiado plana
- Ignora patrones verdaderos
- También falla, por simplón

k es una decisión nuestra

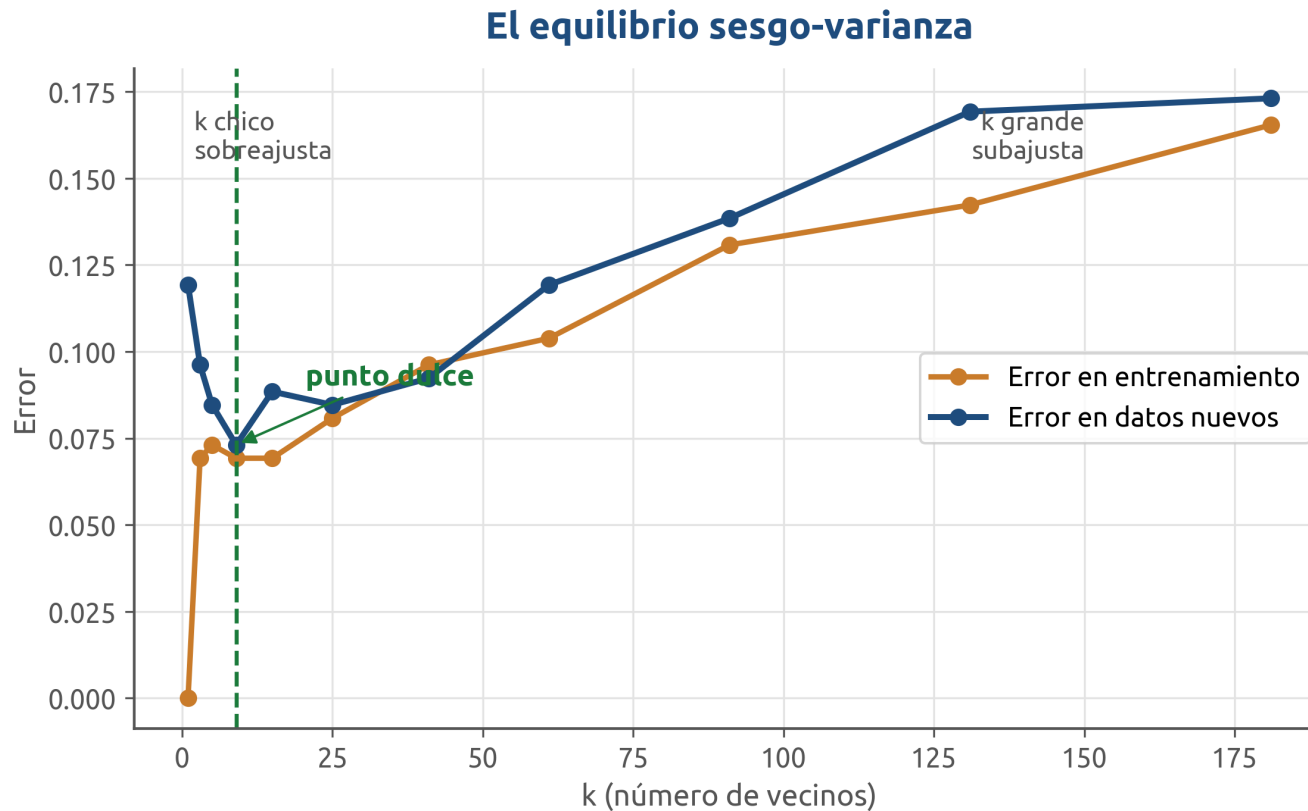
Definición

k es cuántos vecinos consultamos para votar. No lo aprenden los datos: lo elegimos nosotros. Y cambiarlo cambia por completo cómo el modelo separa una clase de otra (su 'frontera de decisión').

En palabras simples

¿Le preguntas solo a 1 amigo o a 100? La respuesta puede cambiar muchísimo según a cuántos consultes.

El equilibrio sesgo-varianza



El error en datos nuevos baja, toca un mínimo y vuelve a subir.

El objetivo es encontrar el valor de k que da el menor error con datos NUEVOS: ni tan chico que memorice, ni tan grande que simplifique de más.

¿Cómo elegimos k ?

Necesitamos probar varios valores de k y quedarnos con el mejor.

Pero NO podemos usar la base de prueba para eso: si la usáramos para elegir k , dejaría de ser 'nueva' y volveríamos a hacer trampa.

En palabras simples

Necesitamos un 'examen de práctica' que salga del material de estudio, sin tocar el examen final.

Afinar k (tuning)

Definición

k es un **HIPERPARÁMETRO**: una perilla que ajustamos ANTES de entrenar. 'Afinar' (tuning) significa probar muchos valores de k con validación cruzada y quedarnos con el que dé mejor desempeño. La lista de valores a probar se llama 'grilla' (grid).

En palabras simples

Como buscar el volumen perfecto de la música: pruebas varios niveles y te quedas con el que mejor se oye.

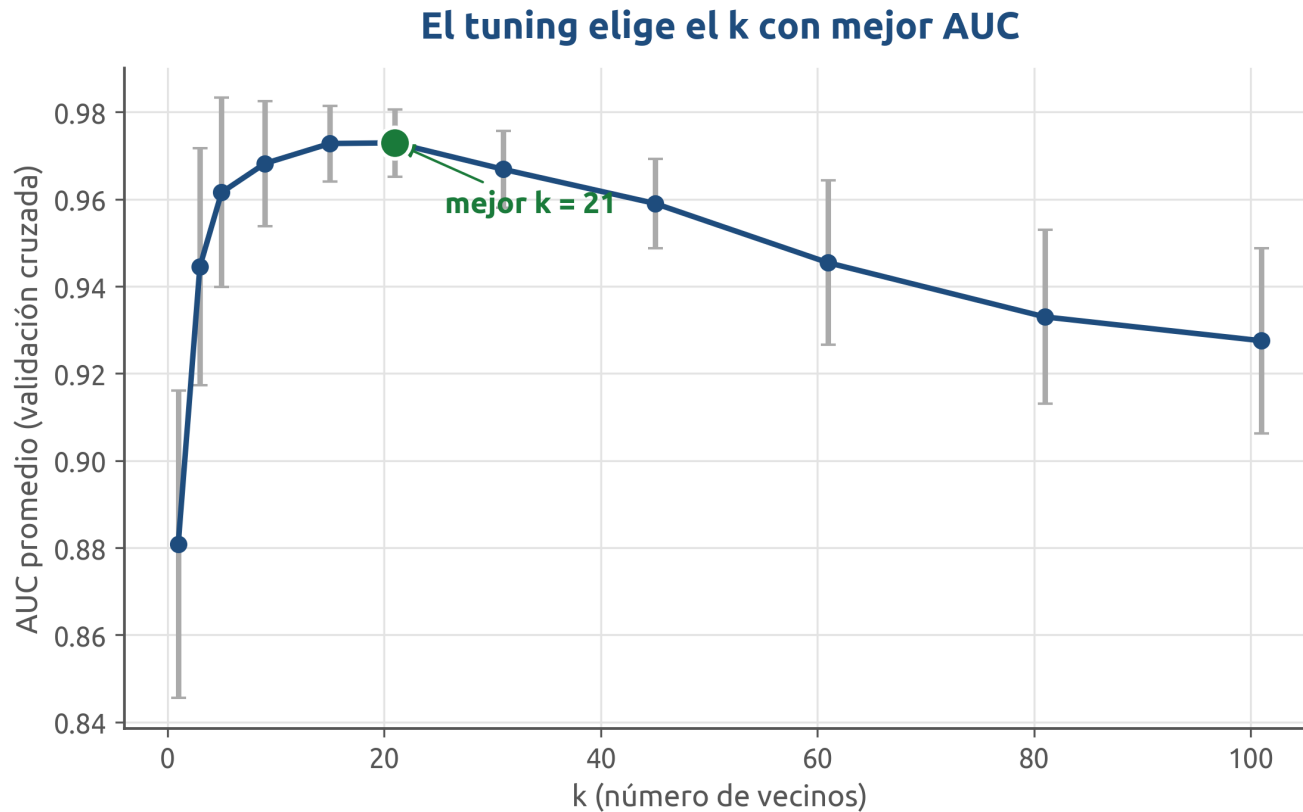
★ Hiperparámetros

Los **hiperparámetros** son las configuraciones de un modelo de Machine Learning que el algoritmo no aprende a partir de los datos, sino que **deben ser definidos** por el analista antes de la construcción del modelo.

Los hiperparámetros se ajustan con un proceso de “Ajuste de hiperparámetros” o ***tuning***, que consiste en buscar y seleccionar los valores adecuados para mejorar su rendimiento.

NOTA: Las regresiones básicas **NO** tienen hiperparámetros. Otros modelos sí (como k-vecinos cercanos).

La curva que decide



Calculamos el desempeño (AUC) para cada k y elegimos el máximo.

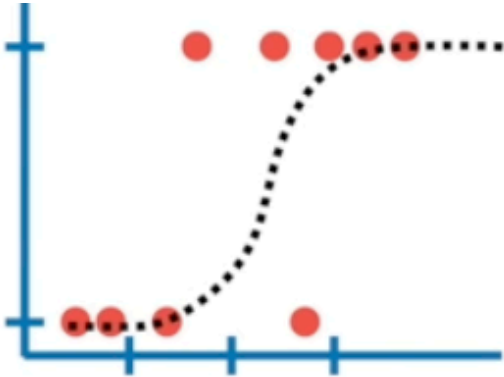
El mejor k cae en medio de la grilla, no en los extremos: justo el equilibrio sesgo-varianza que buscábamos.

Validación cruzada

Validación cruzada

Supongamos que queremos hacer un modelo de clasificación:

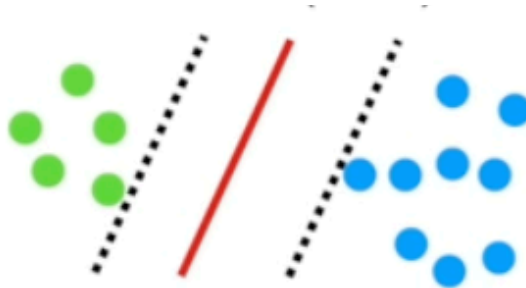
Podríamos usar una regresión logística



O k-vecinos Cercanos



O el algoritmo de Support Vector Machines

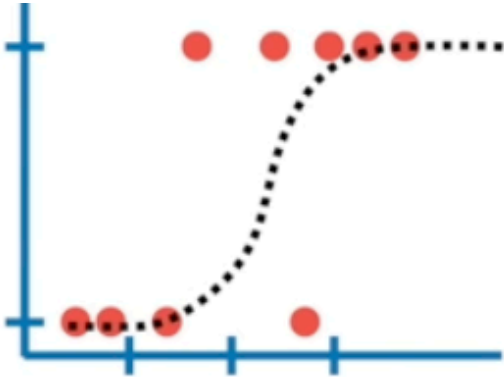


La **validación cruzada** nos permite comparar diferentes métodos de machine Learning para entender como se comportarían en la práctica.

Validación cruzada

Supongamos que queremos hacer un modelo de clasificación:

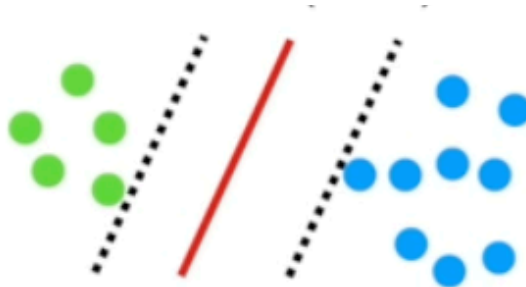
Podríamos usar una regresión logística



O k-vecinos Cercanos



O el algoritmo de Support Vector Machines



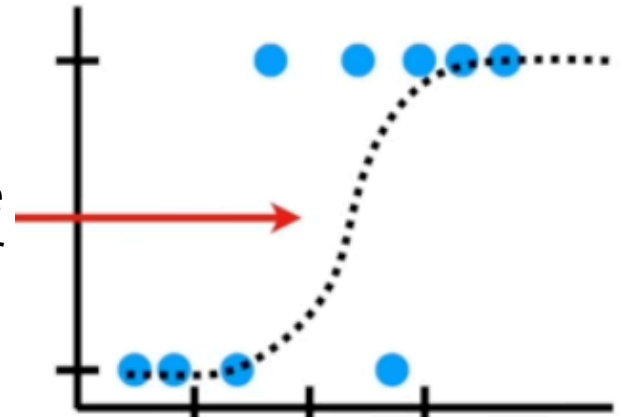
La **validación cruzada** nos permite comparar diferentes métodos de machine Learning para entender como se comportarían en la práctica.

Validación cruzada

← Necesitamos hacer dos cosas con los datos:

- 1) **Estimar** los parámetros del método de machine learning
- 2) **Evaluar** que tan bien funciona el modelo de Machine Learning

A la evaluación de un modelo de Machine Learning, también se le puede decir “probar el algoritmo”.



Validación cruzada



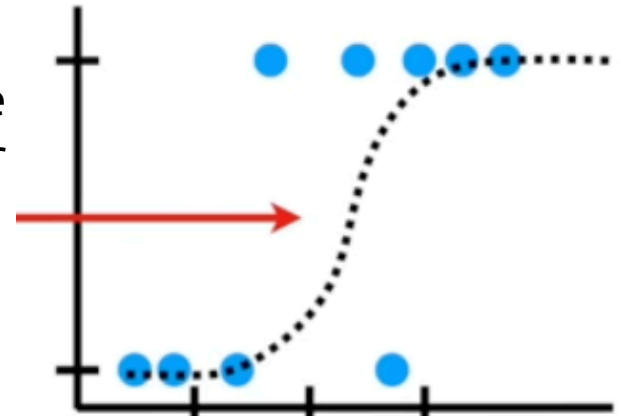
Necesitamos hacer dos cosas con los datos:

- 1) **Estimar** los parámetros del método de machine learning
- 2) **Evaluar** que tan bien funciona el modelo de Machine Learning

A la evaluación de un modelo de Machine Learning, también se le puede decir “probar el algoritmo”.

En resumen, en ML necesitamos:

- 1) **Entrenar** el algoritmo
- 2) **Evaluar o probar** el algoritmo

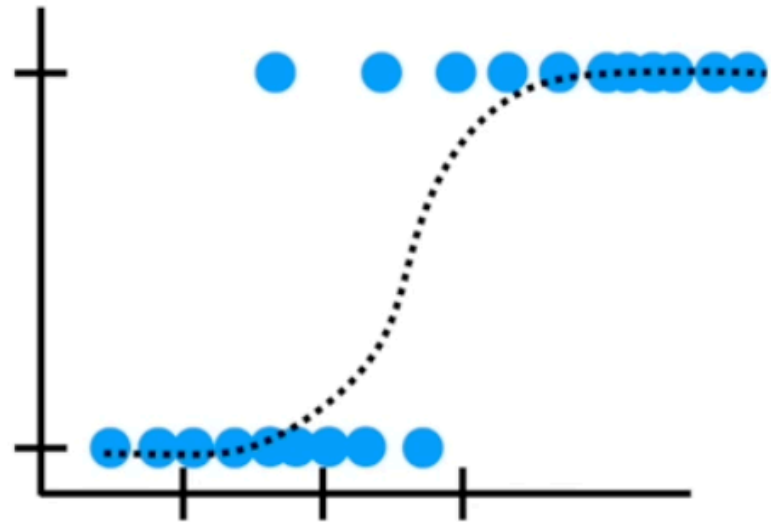


Validación cruzada

No podemos usar los mismos datos para entrenamiento y prueba porque necesitamos saber como se va a comportar el método con datos distintos a aquellos con los que se entrenó.



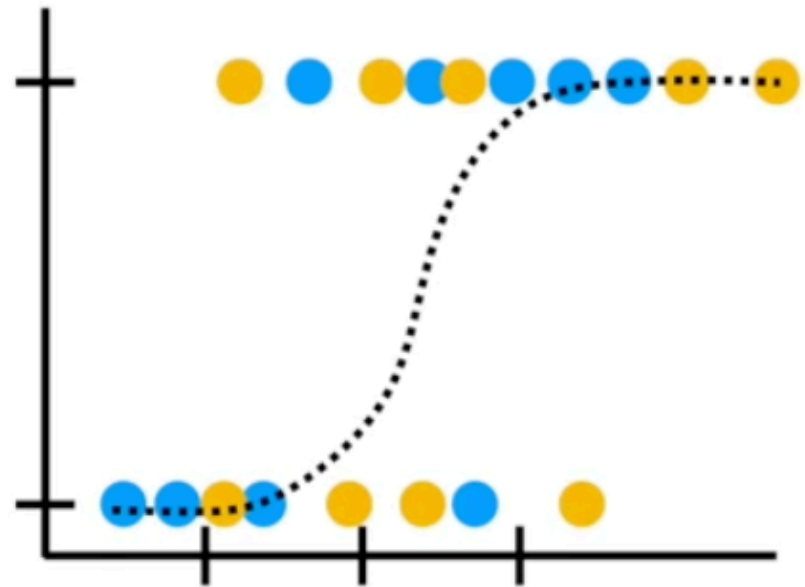
Es como darle las respuestas del examen para que se lo memorice.



Validación cruzada

El enfoque tradicional al iniciar en ML entrenar el modelo con el 75% de los datos...

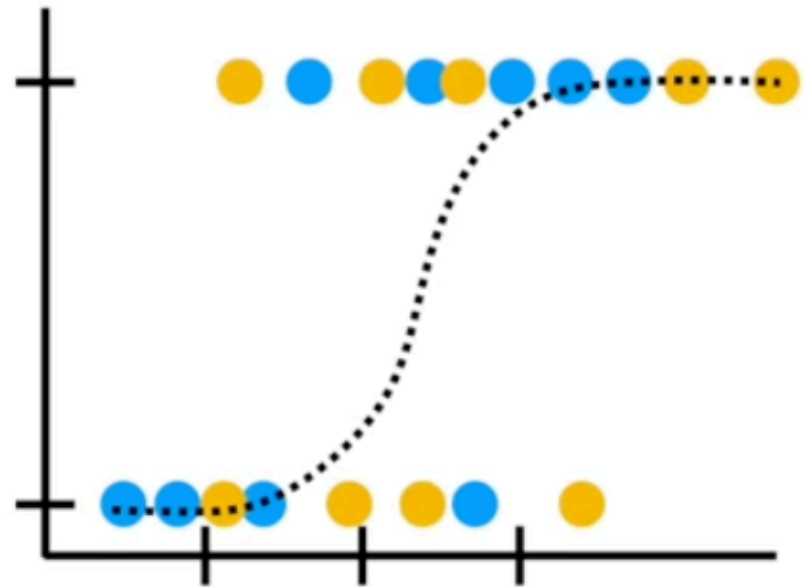
...Y usar el 25% restante para la evaluación del modelo...



Validación cruzada

El enfoque tradicional al iniciar en ML entrenar el modelo con el 75% de los datos...

...Y usar el 25% restante para la evaluación del modelo...



Validación cruzada



Pero ¿cómo sabemos que usar el 75% de los datos para el entrenamiento y el 25% restante es la mejor manera de dividir el dataset?

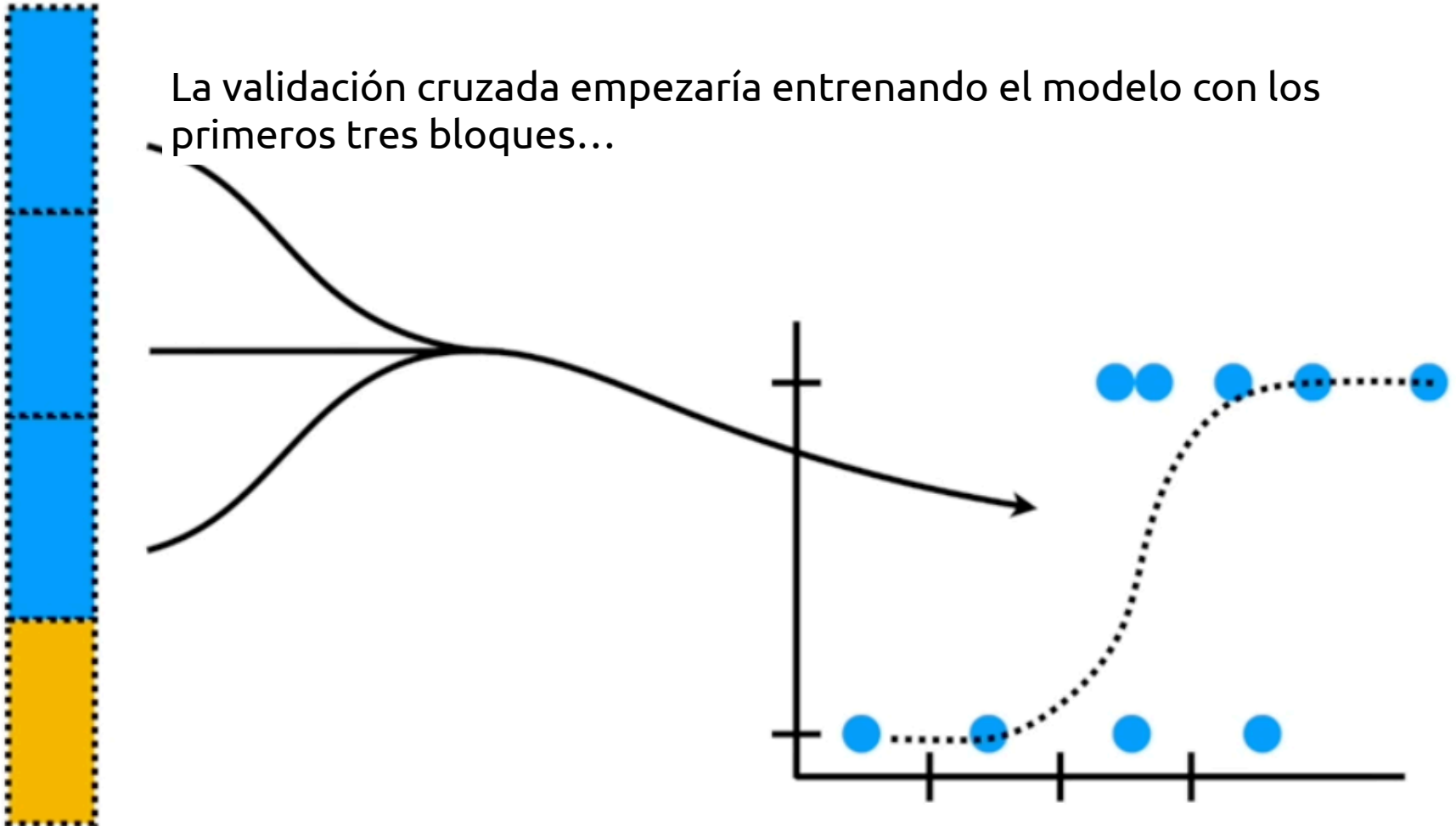
En vez de preocuparnos sobre qué bloque de datos usar cuando dividimos los datos en cachos de 25%, la validación cruzada los usa todos, uno a la vez, y resume los resultados al final.



Validación cruzada

Supongamos que dividimos los datos en cuatro cachos iguales. **Entrenamos** el modelo con los primeros tres cachos (75%) y **evaluamos** el modelo con el cachito restante (25%)

La validación cruzada empezaría entrenando el modelo con los primeros tres bloques...

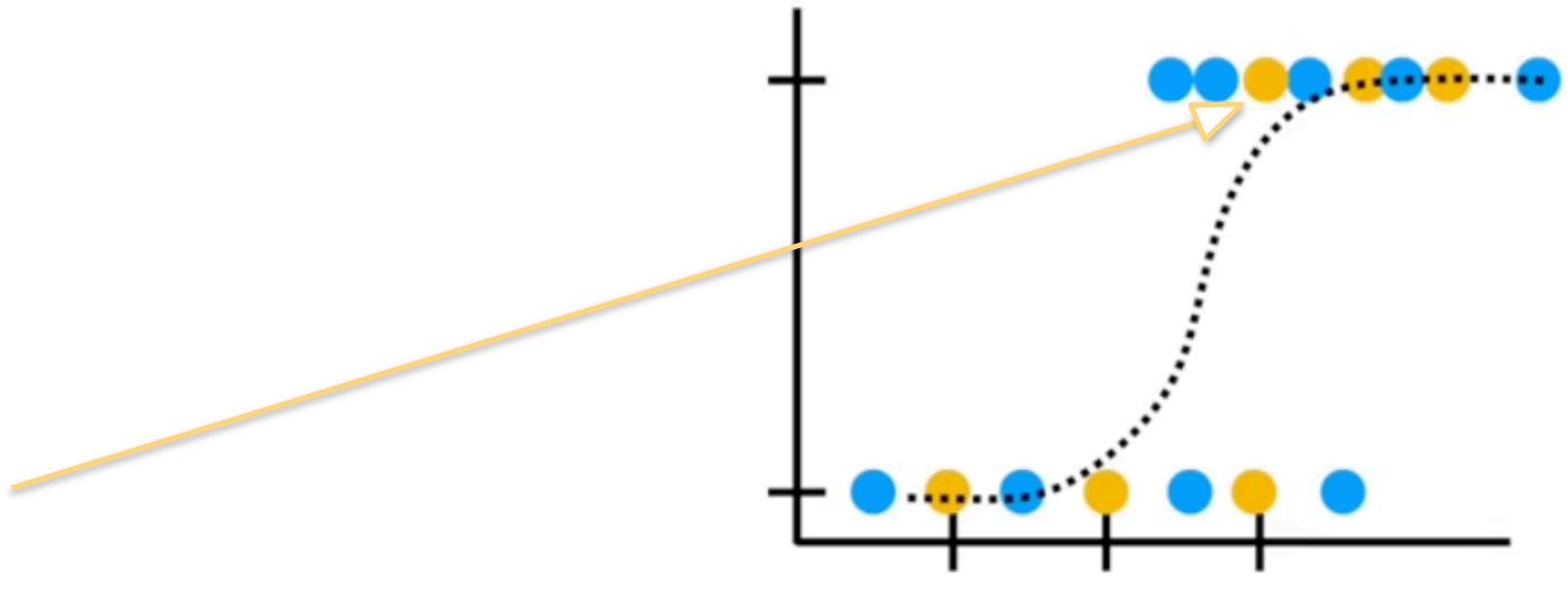


Validación cruzada

Y utilizaría el último bloque de 25% para la evaluación.

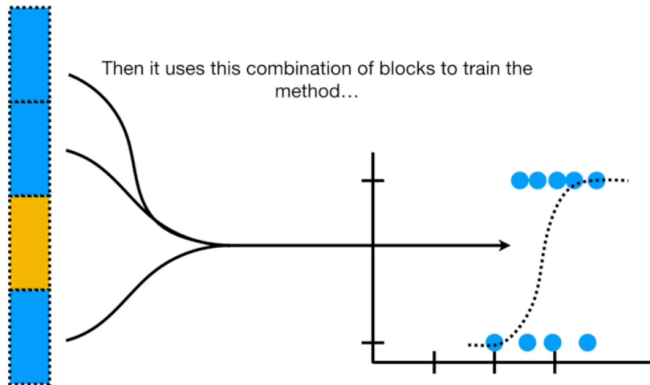
Y mantendría el registro de los resultados de la prueba correspondiente

Test data categorization...	
Correct	Incorrect
5	1



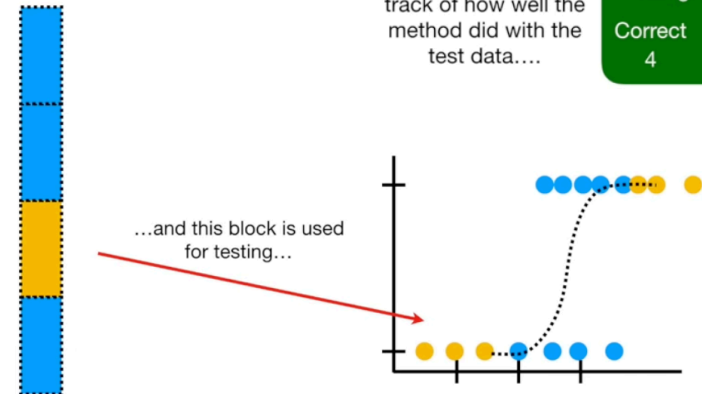
Validación cruzada

Lo mismo para el **bloque 2**

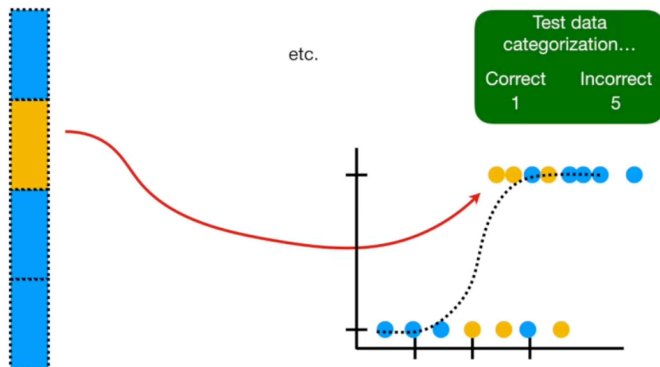


...and then it keeps track of how well the method did with the test data...

Test data categorization...	
Correct	Incorrect
4	2

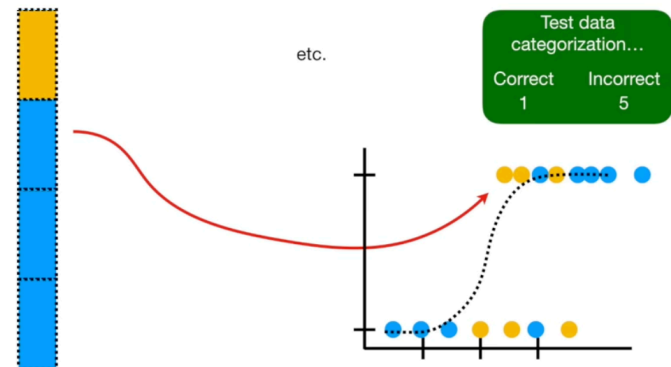


Y para el **bloque 3**



Test data categorization...	
Correct	Incorrect
1	5

Y para el **bloque 4**

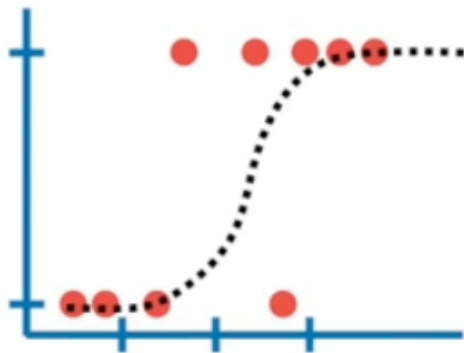


Test data categorization...	
Correct	Incorrect
1	5

Validación cruzada

Al final, cada bloque de datos es usado para en sus respectivas pruebas y así podemos comparar los métodos viendo que también se desempeñaron en estas:

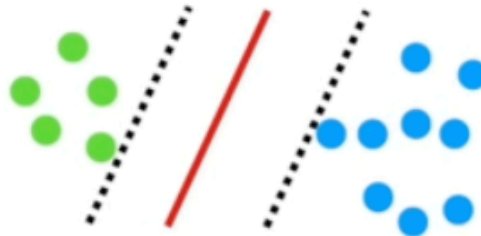
Logistic Regression



Correct
16

Incorrect
8

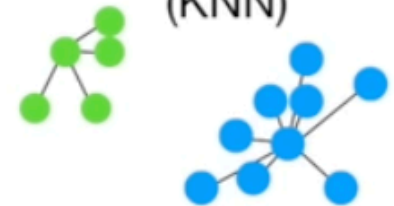
Support Vector machines (SVM)



Correct
18

Incorrect
6

K-nearest neighbors (KNN)



Correct
10

Incorrect
12

Validación cruzada

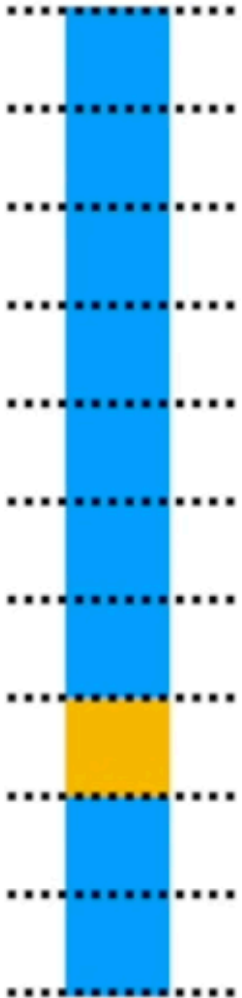
Nota: En este ejemplo, dividimos los datos en 4 bloques.

A esto se le llama “**Validación cruzada de 4 pliegues (Four-Fold Cross Validation)**”.

Sin embargo, el número de bloques o pliegues puede ser cualquiera



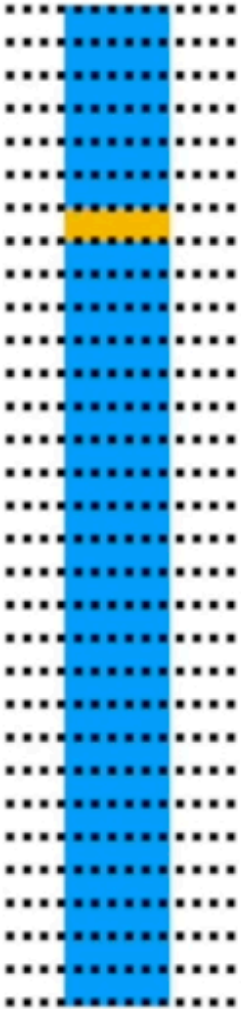
Validación cruzada



En la práctica, es también muy común dividir los datos en 10 bloques.

A esto se le llama **Validación cruzada de 10 pliegues** (Ten-Fold Cross Validation)

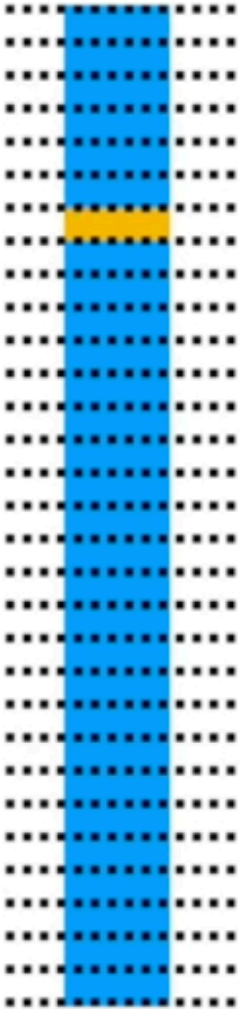
Validación cruzada



El caso más extremo es que cada individuo sea un bloque individual.

A esto se le llama “**Leave One Out Cross Validation (LOOCV)**” y es el método más demandante computacionalmente.

Validación cruzada



La validación cruzada nos sirve para:

1. Afinar las estimaciones de los errores del modelo en la fase de evaluación.
2. Tunear los hiperparámetros
3. Mejorar las comparaciones entre modelos
4. Tunear los hiperparámetros para conseguir un mejor modelo
5. Obtener los valores del umbral de decisión que mejoran el desempeño del modelo

Workflows

Workflows - Juntando todo

Mientras el workflow está "vacío" (no se ha llamado `fit()`), guarda **especificaciones**: la receta sin preparar y el modelo sin ajustar.

Cuando llamas `fit(wf, data = training)`, *tidymodels* hace dos cosas en cadena y las guarda dentro del mismo objeto:

- * (1) ejecuta `prep()` sobre la receta usando esos datos, aprendiendo medianas, sd, umbrales de correlación, niveles de los factores, etc., y
- * (2) ejecuta `bake()` para producir la matriz de diseño y ajusta el modelo sobre ella.

El resultado es un workflow "fitted" o ajustado, que contiene **la receta preparada y el modelo entrenado**, listo para `predict()`.

EL WORKFLOW ES LA UNIDAD MÍNIMA DE TRABAJO EN TIDYMODELS, y es la forma en que trabajaremos de ahora en adelante.

Workflows - Juntando todo

Supongamos que queremos probar 2 modelos distintos; una regresión logística y un k-vecinos cercanos (knn). En ese caso, añadiríamos un paso adicional:

Paso 1: Declaramos los modelos:

```
# Regresión logística (la que ya tenías)
```

```
modelo_logistico <- logistic_reg() %>%  
  set_engine("glm") %>%  
  set_mode("classification")
```

```
# KNN — neighbors lo dejamos tuneable
```

```
modelo_knn <- nearest_neighbor(  
  neighbors      = tune(),          # hiperparámetro a buscar  
  weight_func    = "rectangular",  
  dist_power     = 2                # distancia euclidiana  
) %>%  
  set_engine("kknn") %>%  
  set_mode("classification")
```

Workflows - Juntando todo

Paso 2. Generamos un objeto *workflow()*

Workflow con el modelo logístico

```
wf_logistico <- workflow() %>%  
  add_recipe(receta_reservas) %>%  
  add_model(modelo_logistico)
```

Workflow con el knn

```
wf_knn <- workflow() %>%  
  add_recipe(receta_reservas) %>%  
  add_model(modelo_knn)
```

Workflows - Juntando todo

Un **workflow** es un objeto contenedor de tidymodels que empaqueta en una sola pieza los componentes de un pipeline de modelado: el preprocesador (una recipe o una fórmula) y el modelo (una especificación de parsnip).

Mecánicamente, el objeto tiene tres "slots":

```
wf <- workflow() %>%  
  add_recipe(receta_reservas) %>% # slot "pre" -> preprocesamiento  
  add_model(modelo_knn)           # slot "fit" -> modelo
```

Código

nearest_neighbor()

Especifica el modelo KNN: cuántos vecinos, con qué motor y en qué modo.

Uso en el ejercicio

```
modelo_knn_tune <-  
  nearest_neighbor(neighbors = tune(),  
                  weight_func = "rectangular") %>%  
  set_engine("kknn") %>%  
  set_mode("classification")
```

Qué hace

- nearest_neighbor() declara el modelo. neighbors = el número de vecinos k; tune() significa 'a determinar por validación cruzada'.
- weight_func = "rectangular": voto mayoritario no ponderado (KNN clásico), para ver el equilibrio sesgo-varianza.
- set_engine("kknn") elige el paquete que ejecuta el modelo; set_mode("classification") fija que es clasificación.

workflow()

Empaqueta la receta y el modelo en un solo objeto, robusto y reutilizable.

Uso en el ejercicio

```
workflow_knn_tune <- workflow() %>%  
  add_recipe(receta_creditos) %>%  
  add_model(modelo_knn_tune)
```

Qué hace

- workflow() crea el contenedor. add_recipe() le pega el preprocesamiento y add_model() el modelo.
- Ventaja clave: al ajustar, hace internamente prep + bake + fit; al predecir, aplica el MISMO preprocesamiento a los datos nuevos.
- Evita el error común de olvidar transformar la base de prueba.

fit() / predict()

Entrena el workflow sobre el entrenamiento y predice sobre datos nuevos.

Uso en el ejercicio

```
workflow_knn_fijo_fit <- workflow_knn_fijo %>%  
  fit(data = creditos_entrenamiento)  
  
predict(workflow_knn_fijo_fit, creditos_prueba,  
        type = "class")    # clase predicha  
predict(workflow_knn_fijo_fit, creditos_prueba,  
        type = "prob")     # probabilidades
```

Qué hace

- fit() ajusta todo el workflow (receta + modelo) con los datos de entrenamiento.
- predict(type = "class") devuelve la clase; type = "prob" devuelve las probabilidades de cada clase (útiles para ROC/AUC).
- No hace falta 'hornear' la prueba a mano: el workflow lo hace.

vfold_cv()

Crea los pliegues (folds) de validación cruzada sobre el entrenamiento.

Uso en el ejercicio

```
set.seed(2026)
folds_cv <- vfold_cv(data = creditos_entrenamiento,
                     v = 5,
                     strata = aprobado)
```

Qué hace

- Parte el entrenamiento en $v = 5$ porciones. En cada ronda usa 4 para entrenar y 1 para validar; rota hasta que cada fold valida una vez.
- Sirve para estimar el desempeño SIN tocar la prueba.
- `strata = aprobado` mantiene la proporción de clases en cada fold.

grid_regular() + tune()

Define la lista de valores candidatos del hiperparámetro a probar.

Uso en el ejercicio

```
# en el modelo: neighbors = tune() marca el parámetro a buscar
grilla_k <- grid_regular(neighbors(range = c(1, 101)),
                        levels = 12)
```

Qué hace

- tune() (en el modelo) es un marcador: 'este valor no lo fijo yo, se elige con datos'.
- neighbors(range = c(1, 101)) describe el rango razonable de k.
- grid_regular(..., levels = 12) genera 12 valores uniformes dentro de ese rango: la grilla que el tuning recorrerá.

tune_grid()

Corre la validación cruzada para cada valor de k y guarda sus métricas.

Uso en el ejercicio

```
resultados_tuning <- workflow_knn_tune %>%  
  tune_grid(resamples = folds_cv,  
            grid = grilla_k,  
            metrics = metricas_tuning)
```

Qué hace

- Para CADA k de la grilla, ajusta el workflow en los 5 folds y promedia las métricas. Es el corazón del tuning.
- resamples = los folds; grid = los k a probar; metrics = qué medir (definido con metric_set).
- Devuelve una tabla de resultados por k, lista para comparar.

collect_metrics() + select_best()

Resume las métricas por k y elige el mejor según el criterio que pidamos.

Uso en el ejercicio

```
collect_metrics(resultados_tuning)

mejor_k <- resultados_tuning %>%
  select_best(metric = "roc_auc")
```

Qué hace

- collect_metrics() junta y promedia las métricas de todos los folds, una fila por k: útil para graficar la curva AUC vs k.
- select_best(metric = "roc_auc") devuelve el k con mejor AUC promedio.
- Aquí el mejor k cae DENTRO de la grilla (equilibrio sesgo-varianza), no en sus extremos.

finalize_workflow() + last_fit()

Injecta el k ganador, reentrena con todo el entrenamiento y evalúa en la prueba.

Uso en el ejercicio

```
workflow_knn_final <- workflow_knn_tune %>%  
  finalize_workflow(mejor_k)  
  
ajuste_final <- workflow_knn_final %>%  
  last_fit(split = creditos_split,  
           metrics = metricas_tuning)
```

Qué hace

- finalize_workflow(mejor_k) fija el hiperparámetro tune() con el valor elegido.
- last_fit() hace, en un solo paso: reentrena con TODO el entrenamiento y evalúa en la prueba reservada.
- Es la estimación honesta del desempeño en datos nunca vistos.

metric_set() + roc_auc() + conf_mat()

Definen y calculan las métricas de evaluación de la clasificación.

Uso en el ejercicio

```
metricas_tuning <- metric_set(roc_auc, accuracy,  
                             sensitivity, yardstick::spec)  
  
roc_auc(predicciones_finales, truth = aprobado, .pred_no)  
conf_mat(predicciones_finales, truth = aprobado,  
         estimate = .pred_class)
```

Qué hace

- metric_set() agrupa varias métricas en un solo evaluador reutilizable.
- roc_auc() resume, con un número (0.5 = azar, 1 = perfecto), qué tan bien el modelo ordena los casos por probabilidad.
- conf_mat() arma la matriz de confusión: aciertos y errores por clase.

Gracias